Exercício 1

Objetivo

Criar uma estrutura genérica capaz de armazenar qualquer tipo de entidade e permitir filtragem baseada em predicado.

Enunciado

Você deve criar uma classe genérica chamada:

```
Gerenciador<T>
```

Ela deve:

1. Armazenar itens internamente em uma `List<T>`
2. Permitir adicionar novos itens
3. Retornar todos os itens
4. Permitir filtrar itens com base em uma condição

Requisitos

A classe deve ter:

```
void Adicionar(T item);
List<T> ObterTodos();
List<T> Filtrar(Func<T, bool> criterio);
```

Crie também duas classes de domínio:

```
Aluno
Produto
```

Aluno deve ter:

- Nome
- NotaFinal

Produto deve ter:

- Nome
- Preco

O programa principal deve:

1. Criar um `Gerenciador<Aluno>`
2. Adicionar pelo menos 3 alunos
3. Filtrar alunos com nota maior que 7

Depois:

4. Criar um `Gerenciador<Produto>`
5. Adicionar pelo menos 3 produtos
6. Filtrar produtos com preço maior que 100

Exercício 2

Objetivo

Implementar um repositório genérico que funcione apenas para entidades que possuam Id.

Aqui o aluno começa a trabalhar com **constraints (`where`)**.

Enunciado

Crie uma interface:

```
IEntidade
```

Ela deve conter:

```
int Id { get; set; }
```

Agora crie duas classes que implementem essa interface:

```
Cliente
Pedido
```

Agora implemente:

```
Repository<T>
```

Com a seguinte constraint:

```
where T : IEntidade
```

O Repository deve conter:

```
void Adicionar(T entidade);
T? ObterPorId(int id);
List<T> ObterTodos();
void Remover(int id);
```

Regras

- O repositório deve usar `List<T>` internamente
- `ObterPorId` deve localizar pelo Id
- `Remover` deve remover pelo Id
- Não pode usar cast
- Deve funcionar tanto para Cliente quanto para Pedido

No Program:

1. Criar `Repository<Cliente>`
2. Adicionar 3 clientes
3. Buscar por Id
4. Remover um cliente

Depois:

5. Criar `Repository<Pedido>`
6. Repetir os testes